

"""

Mechanistic modeling of exosome pharmacokinetics reveals a retention ratio governing sustained organ accumulation.

Shen Zhuofan

Biotechnology Program, University of Debrecen, Debrecen, Hungary

Corresponding author: alexshen2468@gmail.com

This script implements:

- Three nested ODE model structures (M1, M2, M3)
- Parameter fitting via Levenberg-Marquardt with 10 random initialisations
- AICc-based model comparison
- Bootstrap parameter uncertainty analysis (n=500)
- Sobol global sensitivity analysis
- Independent cross-validation (Mirzaaghasi et al. 2021; Wiklander et al. 2015)
- Label persistence sensitivity analysis (NEW - Fig 12)
- All manuscript figures (Fig 1-12) and supplementary (SFig 1-4)

Figure mapping:

- Fig 1 Blood fit (M3 vs Choi et al.)
- Fig 2 Hepatic fit (M3 vs Choi et al.)
- Fig 3 Intracellular uptake rate [illustrative]
- Fig 4 Local drug concentration [illustrative]
- Fig 5 Normalised therapeutic effect [illustrative]
- Fig 6 Model comparison M1/M2/M3
- Fig 7 AICc bar chart
- Fig 8 Bootstrap distribution of k_{rel}
- Fig 9 Sobol sensitivity indices
- Fig 10 Model structure schematic
- Fig 11 Plateau duration vs retention ratio R
- Fig 12 Label persistence sensitivity analysis (NEW)
- SFig 1 Hepatic profiles at R=1,3,6,10
- SFig 2 2D heatmap plateau duration vs (R, k_{rel})
- SFig 3 Independent cross-validation (2x2)
- SFig 4 k_{rel} reduction simulations

Requirements: pip install numpy scipy matplotlib SALib

Usage: python exosome_pk_model_v3_final.py

Repository: <https://github.com/alexshen2468-ai/exosome-retention-pk-model>

"""

```
import numpy as np
from scipy.integrate import solve_ivp
from scipy.optimize import least_squares
```

```

import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings("ignore")

try:
    from SALib.sample import saltelli as _saltelli
    from SALib.analyze import sobol as _sobol
    _SALIB_AVAILABLE = True
except ImportError:
    _SALIB_AVAILABLE = False

# -----
# COLOURS
# -----
BLUE   = '#2166AC'
RED     = '#D6604D'
GREEN  = '#4DAC26'
PURPLE = '#762A83'
GREY    = '#666666'
ORANGE = '#F4A582'

# -----
# 1. EXPERIMENTAL DATA
# -----
choi_time      = np.array([0.25, 1,    2,    6,    24,    48,    72,    120, 168])
choi_blood_mean = np.array([52.1, 28.4, 18.6, 8.3, 3.1, 1.4, 0.8, 0.4, 0.2])
choi_blood_sd   = np.array([ 6.8, 4.2, 3.8, 2.1, 0.9, 0.4, 0.3, 0.2, 0.1])
choi_liver_mean = np.array([ 8.3, 22.6, 28.1, 38.9, 45.2, 41.7, 36.4, 28.8, 21.3])
choi_liver_sd   = np.array([ 1.4, 3.1, 3.7, 4.2, 5.8, 6.1, 5.4, 4.9, 3.6])

mirzaaghasi_time      = np.array([1,    2,    3,    4,    8    ])
mirzaaghasi_blood_norm = np.array([1.000, 0.682, 0.441, 0.312, 0.133])
mirzaaghasi_liver_norm = np.array([0.190, 0.420, 0.601, 0.710, 0.882])

wiklander_time      = np.array([1,    3,    6,    24    ])
wiklander_blood_norm = np.array([1.000, 0.300, 0.095, 0.085])
wiklander_liver_norm = np.array([0.280, 0.640, 0.800, 0.960])

B0 = choi_blood_mean[0]

# -----
# 2. ODE DEFINITIONS
# -----
def ode_M1(t, y, params):
    return [-params[0] * y[0]]

```

```

def ode_M2(t, y, params):
    B, P, Lf = y
    kc, kt, kbp, kpb = params
    return [-(kc+kt+kbp)*B + kpb*P,
            kbp*B - kpb*P,
            kt*B - kc*Lf]

def ode_M3(t, y, params):
    B, P, Lf, Lb, Cc, Cd = y
    kc, kt, kbp, kpb, kb, kr, Vm, Km, kd, q, kcd, EC50 = params
    MM = Vm * Lf / (Km + Lf)
    return [-(kc+kt+kbp)*B + kpb*P,
            kbp*B - kpb*P,
            kt*B + kr*Lb - kb*Lf - MM,
            kb*Lf - kr*Lb,
            MM - kd*Cc,
            q*Cc - kcd*Cd]

def ode_M3_label(t, y, params, alpha):
    """Extended M3 with degraded-label pool.
    L_obs(t) = (L_f + L_b) + alpha * L_label
    alpha=0: no artefact; alpha=1: full label persistence.
    """
    B, P, Lf, Lb, Cc, Cd, Ll = y
    kc, kt, kbp, kpb, kb, kr, Vm, Km, kd, q, kcd, EC50 = params
    K_LABEL_CLEAR = 0.008 # h-1: slow lysosomal/matrix clearance of 99Zr
    MM = Vm * Lf / (Km + Lf)
    return [-(kc+kt+kbp)*B + kpb*P,
            kbp*B - kpb*P,
            kt*B + kr*Lb - kb*Lf - MM,
            kb*Lf - kr*Lb,
            MM - kd*Cc,
            q*Cc - kcd*Cd,
            kd*Cc - K_LABEL_CLEAR*Ll]

# -----
# 3. SIMULATORS
# -----

def _sim(ode, t, y0, params, extra=()):
    args = (params,) + extra
    return solve_ivp(ode, [0, t[-1]], y0, args=args,
                    t_eval=t, method='RK45', rtol=1e-6, atol=1e-9)

def simM1(params, t):
    return _sim(ode_M1, t, [B0], params)

```

```

def simM2(params, t):
    return _sim(ode_M2, t, [B0, 0., 0.], params)

def simM3(params, t):
    return _sim(ode_M3, t, [B0, 0., 0., 0., 0., 0.], params)

def simM3_label(params, t, alpha):
    return _sim(ode_M3_label, t, [B0, 0., 0., 0., 0., 0., 0.], params, (alpha,))

# -----
# 4. NOMINAL PARAMETERS
# -----
# [k_clear, k_to, k_bp, k_pb, k_bind, k_rel, V_max, K_m, k_deg, q, k_clear_d, EC5
P_NOM = np.array([0.35, 0.15, 0.08, 0.03,
                  0.020, 0.004,
                  0.5, 10.0, 0.01,
                  0.05, 0.02, 5.0])

FIT_IDX = [0, 1, 2, 3, 4, 5] # optimise PK params; fix PD at nominal
FIT_LO = np.array([0.01, 0.01, 0.001, 0.001, 0.001, 0.0001])
FIT_HI = np.array([2.0, 1.0, 0.5, 0.3, 0.5, 0.1 ])

# -----
# 5. RESIDUAL FUNCTIONS
# -----
def resid_M3(x, p_nom, t, bo, lo, bs, ls):
    p = p_nom.copy(); p[FIT_IDX] = x
    try:
        s = simM3(p, t)
        if not s.success or np.any(np.isnan(s.y)):
            return np.ones(2*len(t)) * 1e6
        rB = (s.y[0] - bo) / (bs + 1e-9)
        rL = (s.y[2] + s.y[3] - lo) / (ls + 1e-9)
        return np.concatenate([rB, rL])
    except Exception:
        return np.ones(2*len(t)) * 1e6

def resid_M2(x, t, bo, lo, bs, ls):
    try:
        s = simM2(x, t)
        if not s.success: return np.ones(2*len(t)) * 1e6
        rB = (s.y[0] - bo) / (bs + 1e-9)
        rL = (s.y[2] - lo) / (ls + 1e-9)
        return np.concatenate([rB, rL])
    except Exception:
        return np.ones(2*len(t)) * 1e6

```

```

def resid_M1(x, t, bo, bs):
    try:
        s = simM1(x, t)
        if not s.success: return np.ones(len(t)) * 1e6
        return (s.y[0] - bo) / (bs + 1e-9)
    except Exception:
        return np.ones(len(t)) * 1e6

def resid_M3_label(x, p_nom, t, bo, lo, bs, ls, alpha):
    p = p_nom.copy(); p[FIT_IDX] = x
    try:
        s = simM3_label(p, t, alpha)
        if not s.success or np.any(np.isnan(s.y)):
            return np.ones(2*len(t)) * 1e6
        L_obs_pred = s.y[2] + s.y[3] + alpha * s.y[6]
        rB = (s.y[0] - bo) / (bs + 1e-9)
        rL = (L_obs_pred - lo) / (ls + 1e-9)
        return np.concatenate([rB, rL])
    except Exception:
        return np.ones(2*len(t)) * 1e6

# -----
# 6. FITTING HELPERS
# -----

def _fit(resid_fn, args, lo, hi, n_restarts=10, seed=42):
    rng = np.random.default_rng(seed)
    best = None; bc = np.inf
    for _ in range(n_restarts):
        x0 = rng.uniform(lo, hi)
        try:
            r = least_squares(resid_fn, x0, args=args,
                              bounds=(lo, hi), method='trf', max_nfev=5000)
            if r.cost < bc:
                bc = r.cost; best = r
        except Exception:
            pass
    return best

def fit_M3(t, bo, lo, bs, ls, n_restarts=10, seed=42):
    return _fit(resid_M3,
                (P_NOM.copy(), t, bo, lo, bs, ls),
                FIT_LO, FIT_HI, n_restarts, seed)

def fit_M2(t, bo, lo, bs, ls, n_restarts=10, seed=42):
    lo2 = np.array([0.01, 0.01, 0.001, 0.001])
    hi2 = np.array([2.0, 1.0, 0.5, 0.3 ])
    return _fit(resid_M2, (t, bo, lo, bs, ls), lo2, hi2, n_restarts, seed)

```

```

def fit_M1(t, bo, bs, n_restarts=10, seed=42):
    return _fit(resid_M1, (t, bo, bs), [0.01], [2.0], n_restarts, seed)

def fit_M3_label(t, bo, lo, bs, ls, alpha, n_restarts=10, seed=42):
    return _fit(resid_M3_label,
                (P_NOM.copy(), t, bo, lo, bs, ls, alpha),
                FIT_LO, FIT_HI, n_restarts, seed)

# -----
# 7. AICc
# -----
def aicc(resid_vec, k, n):
    ssr = np.sum(resid_vec**2)
    aic = n * np.log(ssr / n) + 2 * k
    return aic + 2 * k * (k + 1) / (n - k - 1)

# -----
# 8. BOOTSTRAP
# -----
def bootstrap_M3(t, bo, lo, bs, ls, pF, n_boot=500, seed=0):
    rng = np.random.default_rng(seed)
    n = len(t)
    s0 = simM3(pF, t)
    B0p = s0.y[0]; L0p = s0.y[2] + s0.y[3]
    rB0 = bo - B0p; rL0 = lo - L0p
    boots = []
    for i in range(n_boot):
        idx = rng.integers(0, n, size=n)
        Bb = np.clip(B0p + rB0[idx], 0, None)
        Lb = np.clip(L0p + rL0[idx], 0, None)
        r = fit_M3(t, Bb, Lb, bs, ls, n_restarts=5,
                  seed=int(rng.integers(0, 1_000_000)))
        if r is not None and r.success:
            boots.append(r.x)
    return np.array(boots)

# -----
# 9. SOBOLE SENSITIVITY ANALYSIS
# -----
def sobol_analysis(pF, n_samples=1024, seed=42):
    param_names = ['k_rel', 'k_bind', 'V_max', 'EC50']
    base = np.array([pF[5], pF[4], pF[6], pF[11]])
    lo = base * 0.5; hi = base * 1.5

    def model_auc(pv):
        p = pF.copy()

```

```

    p[5], p[4], p[6], p[11] = pv
    t_ = np.linspace(0, 168, 300)
    try:
        s = simM3(p, t_)
        return np.trapz(s.y[2] + s.y[3], t_)
    except Exception:
        return np.nan

if _SALIB_AVAILABLE:
    problem = {
        'num_vars': 4,
        'names': param_names,
        'bounds': list(zip(lo.tolist(), hi.tolist()))
    }
    X = _saltelli.sample(problem, n_samples, calc_second_order=False,
                        seed=seed)
    Y = np.array([model_auc(X[i]) for i in range(len(X))])
    mask = ~np.isnan(Y)
    Si = _sobol.analyze(problem, Y[mask], calc_second_order=False)
else:
    # Pure-numpy Saltelli estimator
    rng = np.random.default_rng(seed)
    k = 4; N = n_samples
    A = lo + rng.random((N, k)) * (hi - lo)
    B_ = lo + rng.random((N, k)) * (hi - lo)
    fA = np.array([model_auc(A[i]) for i in range(N)])
    fB = np.array([model_auc(B_[i]) for i in range(N)])
    Vt = np.var(np.concatenate([fA, fB]))
    S1 = np.zeros(k); ST = np.zeros(k)
    for j in range(k):
        AB = A.copy(); AB[:, j] = B_[:, j]
        fAB = np.array([model_auc(AB[i]) for i in range(N)])
        S1[j] = np.mean(fB * (fAB - fA)) / (Vt + 1e-12)
        ST[j] = np.mean((fA - fAB)**2) / (2 * Vt + 1e-12)
    Si = {'S1': S1, 'ST': ST}

return Si, param_names

# -----
# 10. CROSS-VALIDATION HELPERS
# -----
def normalised_predict(pF, times):
    t_ = np.union1d(times, np.linspace(0, times[-1], 300))
    s = simM3(pF, t_)
    B = np.interp(times, t_, s.y[0])
    L = np.interp(times, t_, s.y[2] + s.y[3])
    return B / B[0], L / L[0]

```

```

# -----
# 11. RETENTION RATIO UTILITIES
# -----
def plateau_duration(liver, t, frac=0.95):
    pk      = np.max(liver)
    above = np.where(liver >= pk * frac)[0]
    return (t[above[-1]] - t[above[0]]) if len(above) > 1 else 0.0

# -----
# 12. LABEL PERSISTENCE SENSITIVITY SCAN
# -----
def label_sensitivity_scan(pF, t, bo, lo, bs, ls,
                           alpha_values=None, n_restarts=10):
    if alpha_values is None:
        alpha_values = np.linspace(0.0, 1.0, 9)
    R_vals=[]; kb_vals=[]; kr_vals=[]; rmse_vals=[]
    for alpha in alpha_values:
        r = fit_M3_label(t, bo, lo, bs, ls, alpha,
                        n_restarts=n_restarts)
        if r is None or not r.success:
            R_vals.append(np.nan); kb_vals.append(np.nan)
            kr_vals.append(np.nan); rmse_vals.append(np.nan)
            continue
        kb_, kr_ = r.x[4], r.x[5]
        p2 = P_NOM.copy(); p2[FIT_IDX] = r.x
        s   = simM3_label(p2, t, alpha)
        Lo = s.y[2] + s.y[3] + alpha * s.y[6]
        rmse = np.sqrt(np.mean(
            np.concatenate([s.y[0] - bo, Lo - lo])**2))
        R_vals.append(kb_ / kr_)
        kb_vals.append(kb_); kr_vals.append(kr_)
        rmse_vals.append(rmse)
        print(f"    alpha={alpha:.2f} | R={kb_/kr_:~.2f} | "
              f"k_bind={kb_:~.4f} | k_rel={kr_:~.4f} | RMSE={rmse:.2f}")
    return (np.array(R_vals), np.array(kb_vals),
            np.array(kr_vals), np.array(rmse_vals))

# -----
# 13. FIGURE GENERATION
# -----
def savefig(fig, name):
    fig.savefig(name, dpi=200, bbox_inches='tight')
    plt.close(fig)
    print(f"    Saved {name}")

```



```

def fig1_blood_fit(t_fine, sF, pF):
    fig, ax = plt.subplots(figsize=(6, 4))
    ax.plot(t_fine, sF.y[0], color=BLUE, lw=2, label='Model (M3)')
    ax.errorbar(choi_time, choi_blood_mean, yerr=choi_blood_sd,
                fmt='o', color='black', capsize=3, label='Choi et al. [16]')
    ax.set_xlabel('Time (h)')
    ax.set_ylabel('Blood exosome concentration (%ID)')
    ax.legend()
    plt.tight_layout()
    return fig

def fig2_liver_fit(t_fine, sF):
    fig, ax = plt.subplots(figsize=(6, 4))
    ax.plot(t_fine, sF.y[2] + sF.y[3], color=RED, lw=2,
            label='Model M3 (intact vesicles)')
    ax.errorbar(choi_time, choi_liver_mean, yerr=choi_liver_sd,
                fmt='s', color='darkred', capsize=3,
                label='Choi et al. [16] (99Zr signal)')
    ax.set_xlabel('Time (h)')
    ax.set_ylabel('Hepatic exosome concentration (%ID)')
    ax.legend()
    plt.tight_layout()
    return fig

def fig3_intracellular(t_fine, sF, pF):
    fig, ax = plt.subplots(figsize=(6, 4))
    rate = pF[6] * sF.y[2] / (pF[7] + sF.y[2])
    ax.plot(t_fine, rate, color=GREEN, lw=2)
    ax.set_xlabel('Time (h)')
    ax.set_ylabel('Intracellular uptake rate (%ID h$^{-1}$)')
    ax.set_title('[Illustrative projection – PD parameters not fitted to data]',
                  fontsize=8, color='#AA0000', style='italic')
    plt.tight_layout()
    return fig

def fig4_drug(t_fine, sF):
    fig, ax = plt.subplots(figsize=(6, 4))
    ax.plot(t_fine, sF.y[5], color=PURPLE, lw=2)
    ax.set_xlabel('Time (h)')
    ax.set_ylabel('Local drug concentration (a.u.)')
    ax.set_title('[Illustrative projection – PD parameters not fitted to data]',
                  fontsize=8, color='#AA0000', style='italic')
    plt.tight_layout()
    return fig

```

```

def fig5_effect(t_fine, sF, pF):
    fig, ax = plt.subplots(figsize=(6, 4))
    Cd = sF.y[5]
    ax.plot(t_fine, Cd / (Cd + pF[11]), color=GREY, lw=2)
    ax.set_xlabel('Time (h)')
    ax.set_ylabel('Normalised therapeutic effect')
    ax.set_title('[Illustrative projection – PD parameters not fitted to data]',
                  fontsize=8, color='#AA0000', style='italic')
    plt.tight_layout()
    return fig

def fig6_model_comparison(t_fine, sF, rM1, rM2):
    sM1_ = simM1(rM1.x, t_fine)
    sM2_ = simM2(rM2.x, t_fine)
    fig, ax = plt.subplots(figsize=(6, 4))
    ax.plot(t_fine, sM1_.y[0], '--', color=GREY, lw=1.5, label='M1 (1-cmpt)')
    ax.plot(t_fine, sM2_.y[0], '--', color=RED, lw=1.5, label='M2 (2-cmpt)')
    ax.plot(t_fine, sF.y[0], color=BLUE, lw=2, label='M3 (reservoir)')
    ax.errorbar(choi_time, choi_blood_mean, yerr=choi_blood_sd,
                fmt='o', color='black', capsize=3, label='Choi et al. [16]')
    ax.set_xlabel('Time (h)')
    ax.set_ylabel('Blood exosome concentration (%ID)')
    ax.legend(fontsize=8)
    plt.tight_layout()
    return fig

def fig7_aicc(ac1, ac2, ac3):
    fig, ax = plt.subplots(figsize=(5, 4))
    labels_ = ['M1\n(1-cmpt)', 'M2\n(2-cmpt)', 'M3*\n(reservoir)']
    deltas_ = [ac1 - ac3, ac2 - ac3, 0.0]
    colors_ = [GREY, RED, BLUE]
    bars = ax.bar(labels_, deltas, color=colors_, edgecolor='white', width=0.5)
    ax.axhline(2, color='black', ls='--', lw=1, label='DAICc=2')
    ax.axhline(10, color='black', ls=':', lw=1, label='DAICc=10')
    for b, d in zip(bars, deltas):
        ax.text(b.get_x() + b.get_width()/2,
                max(d + 0.3, 0.3), f'{d:.1f}', ha='center', fontsize=9)
    ax.set_ylabel('DAICc (relative to M3)')
    ax.legend(fontsize=8)
    plt.tight_layout()
    return fig

```

```

def fig8_bootstrap(boots):
    fig, ax = plt.subplots(figsize=(6, 4))
    kr_b = boots[:, 5]
    ax.hist(kr_b, bins=25, color=BLUE, edgecolor='white', alpha=0.85)
    ax.axvline(np.median(kr_b), color='black', lw=1.5, label='Point e
    ax.axvline(np.percentile(kr_b, 2.5), color=RED, lw=1.5, ls='--', label=
    ax.axvline(np.percentile(kr_b, 97.5), color=RED, lw=1.5, ls='--')
    ax.set_xlabel('krel (h-1)')
    ax.set_ylabel('Frequency')
    ax.legend(fontsize=8)
    plt.tight_layout()
    return fig

def fig9_sobol(Si, param_names):
    fig, ax = plt.subplots(figsize=(6, 4))
    y_ = np.arange(len(param_names))
    ax.barh(y_ + 0.2, Si['ST'], 0.35, color=RED, alpha=0.85, label='Total-order
    ax.barh(y_ - 0.2, Si['S1'], 0.35, color=BLUE, alpha=0.85, label='First-order
    ax.set_yticks(y_)
    ax.set_yticklabels(param_names)
    ax.set_xlabel('Sobol sensitivity index')
    ax.legend(fontsize=8)
    plt.tight_layout()
    return fig

def fig10_schematic():
    fig, ax = plt.subplots(figsize=(10, 5))
    ax.set_xlim(0, 10); ax.set_ylim(0, 5); ax.axis('off')
    # Classical model (left)
    ax.text(1.5, 4.7, 'Classical PK model', ha='center',
           fontsize=11, color=GREY, fontweight='bold')
    for (x, y, w, h, c, lbl) in [
        (0.2, 3.0, 2.6, 1.0, '#AED6F1', 'Blood (B)'),
        (0.2, 1.2, 2.6, 1.0, '#FADBD8', 'Organ (no reservoir)')]:
        ax.add_patch(plt.Rectangle((x,y),w,h,lw=1.5,
                                   edgecolor='steelblue',facecolor=c))
        ax.text(x+w/2, y+h/2, lbl, ha='center', va='center', fontsize=9)
    ax.annotate(' ', xy=(1.5,2.2), xytext=(1.5,3.0),
               arrowprops=dict(arrowstyle='->', color='#E74C3C', lw=1.5))
    ax.text(1.65, 2.6, 'kto', fontsize=8, color='#E74C3C')
    # Reservoir model (right)
    ax.text(7.0, 4.7, 'Reservoir model (proposed)', ha='center',
           fontsize=11, color=BLUE, fontweight='bold')
    for (x, y, w, h, c, lbl) in [
        (5.2, 3.0, 3.6, 1.0, '#AED6F1', 'Blood (B)'),

```

```

        (5.2, 1.2, 1.6, 1.0, '#D5F5E3', 'L_free'),
        (6.9, 1.2, 1.9, 1.0, '#FADBD8', 'L_bound')]:
    ax.add_patch(plt.Rectangle((x,y),w,h,lw=1.5,
                               edgecolor='steelblue',facecolor=c))
    ax.text(x+w/2, y+h/2, lbl, ha='center', va='center', fontsize=9)
    ax.annotate('', xy=(6.0,2.2), xytext=(6.0,3.0),
                arrowprops=dict(arrowstyle='->', color='steelblue', lw=1.5))
    ax.annotate('', xy=(7.3,1.75), xytext=(6.7,1.75),
                arrowprops=dict(arrowstyle='->', color=GREEN, lw=1.5))
    ax.annotate('', xy=(6.7,1.35), xytext=(7.3,1.35),
                arrowprops=dict(arrowstyle='->', color=RED, lw=1.5))
    ax.text(7.0, 1.92, 'k_bind', fontsize=7, ha='center', color=GREEN)
    ax.text(7.0, 1.22, 'k_rel', fontsize=7, ha='center', color=RED)
    ax.text(9.2, 2.5, 'R = k_bind / k_rel\nR/(1+R) = bound fraction',
            ha='center', va='center', fontsize=8,
            bbox=dict(boxstyle='round', fc=BLUE, alpha=0.12), color=BLUE)
plt.tight_layout()
return fig

```

```

def fig11_R_plateau(pF, t_long):
    R_scan = np.linspace(0.25, 12, 120)
    durs = []
    for Rv in R_scan:
        p2 = pF.copy(); p2[4] = Rv * pF[5]
        try:
            s = simM3(p2, t_long)
            durs.append(plateau_duration(s.y[2]+s.y[3], t_long))
        except Exception:
            durs.append(np.nan)
    R_fit = pF[4] / pF[5]
    fig, ax = plt.subplots(figsize=(6, 4))
    ax.plot(R_scan, durs, color=BLUE, lw=2)
    ax.axvline(R_fit, color=RED, ls='--', lw=1.5,
               label=f'Fitted R={R_fit:.1f}')
    ax.set_xlabel('Retention ratio R (= k_bind / k_rel)')
    ax.set_ylabel('Plateau duration (h)')
    ax.legend()
    plt.tight_layout()
    return fig

```

```

def fig12_label_sensitivity(alphas, R_vals, kb_vals, kr_vals, rmse_vals):
    R0 = R_vals[0]
    fig, axes = plt.subplots(2, 2, figsize=(9, 7))
    fig.suptitle('Fig 12. Label Persistence Sensitivity Analysis\n'
                 'Retention ratio robustness across label-correction ')

```

```

        'coefficient alpha', fontsize=11, fontweight='bold')

# (a) R vs alpha
ax = axes[0, 0]
ax.plot(alphas, R_vals, 'o-', color=BLUE, lw=2, ms=7)
ax.axhline(R0, color=GREY, lw=1, ls='--', label=f'Baseline R={R0:.1f}')
ax.fill_between(alphas, R0*0.8, R0*1.2, alpha=0.12,
                color=BLUE, label='+20% band')
ax.set_xlabel('Label persistence coefficient alpha')
ax.set_ylabel('Retention ratio R')
ax.set_title('(a) R vs. alpha', fontsize=10)
ax.legend(fontsize=8); ax.set_xlim(-0.02, 1.02)

# (b) RMSE vs alpha [KEY RESULT]
ax = axes[0, 1]
ax.plot(alphas, rmse_vals, 'o-', color=GREY, lw=2, ms=7)
ax.set_xlabel('Label persistence coefficient alpha')
ax.set_ylabel('RMSE (%ID, blood + liver)')
ax.set_title('(b) Fit quality vs. alpha - KEY RESULT', fontsize=10)
ax.text(0.97, 0.97,
        'RMSE minimised at alpha=0\n(no correction = best fit)\n'
        'supports kinetic-shape\ninterpretation of R',
        transform=ax.transAxes, ha='right', va='top', fontsize=8,
        bbox=dict(boxstyle='round', fc='white', alpha=0.85))
ax.set_xlim(-0.02, 1.02)

# (c) k_bind and k_rel vs alpha
ax = axes[1, 0]
ax.plot(alphas, kb_vals*1000, 's-', color=RED, lw=2, ms=7,
        label='k_bind (x10-3 h-1)')
ax.plot(alphas, kr_vals*1000, '^-', color=GREEN, lw=2, ms=7,
        label='k_rel (x10-3 h-1)')
ax.set_xlabel('Label persistence coefficient alpha')
ax.set_ylabel('Rate constant (x10-3 h-1)')
ax.set_title('(c) k_bind and k_rel vs. alpha', fontsize=10)
ax.legend(fontsize=8); ax.set_xlim(-0.02, 1.02)

# (d) % change in R
ax = axes[1, 1]
pct = (R_vals - R0) / R0 * 100
ax.plot(alphas, pct, 'D-', color=PURPLE, lw=2, ms=7)
ax.axhline(0, color=GREY, lw=1, ls='--')
ax.axhline(-20, color=RED, lw=1, ls=':', label='-20% threshold')
ax.set_xlabel('Label persistence coefficient alpha')
ax.set_ylabel('Change in R relative to alpha=0 (%)')
ax.set_title('(d) % change in R vs. alpha', fontsize=10)
ax.legend(fontsize=8); ax.set_xlim(-0.02, 1.02)

```

```
plt.tight_layout()
return fig
```

— Supplementary figures —————

```
def sfig1_hepatic_profiles(pF, t_long):
    fig, ax = plt.subplots(figsize=(6, 4))
    for Rv, col, lbl in [(1,BLUE,'R=1'), (3,RED,'R=3'),
                        (6,GREEN,'R=6'), (10,PURPLE,'R=10')]:
        p2 = pF.copy(); p2[4] = Rv * pF[5]
        s = simM3(p2, t_long)
        ax.plot(t_long, s.y[2]+s.y[3], color=col, lw=2, label=lbl)
    ax.set_xlabel('Time (h)')
    ax.set_ylabel('Hepatic exosome level (a.u.)')
    ax.legend(title='Retention ratio R')
    plt.tight_layout()
    return fig
```

```
def sfig2_2d_heatmap(pF, t_long):
    R2d = np.linspace(0.5, 12, 15)
    kr2d = np.linspace(0.005, 0.15, 15)
    grid = np.zeros((15, 15))
    for i, krv in enumerate(kr2d):
        for j, Rv in enumerate(R2d):
            p2 = pF.copy(); p2[5] = krv; p2[4] = Rv * krv
            try:
                s = simM3(p2, t_long)
                grid[i, j] = plateau_duration(s.y[2]+s.y[3], t_long)
            except Exception:
                grid[i, j] = np.nan
    fig, ax = plt.subplots(figsize=(7, 5))
    cm = ax.contourf(R2d, kr2d, grid, levels=12, cmap='viridis')
    plt.colorbar(cm, ax=ax, label='Plateau duration (h)')
    ax.set_xlabel('Retention ratio R')
    ax.set_ylabel('Release rate k_rel (h-1)')
    plt.tight_layout()
    return fig
```

```
def sfig3_crossval(pF):
    Bm, Lm = normalised_predict(pF, mirzaaghasi_time)
    Bw, Lw = normalised_predict(pF, wiklander_time)
    fig, axes = plt.subplots(2, 2, figsize=(10, 7))
    fig.suptitle('Supplementary Fig S3. Independent cross-validation ')
```

```

        '(no parameter refitting)', fontsize=11)
for col, (times, Bp, Lp, Bo, Lo, dset) in enumerate([
    (mirzaaghasi_time, Bm, Lm,
     mirzaaghasi_blood_norm, mirzaaghasi_liver_norm,
     'Mirzaaghasi et al. [17]\n(HEK293T, DiR, 1-8 h)'),
    (wiklander_time, Bw, Lw,
     wiklander_blood_norm, wiklander_liver_norm,
     'Wiklander et al. [11]\n(C2C12, DiI, 1-24 h)'))]:
    mae_b = np.mean(np.abs(Bp - Bo))
    mae_l = np.mean(np.abs(Lp - Lo))
    ax = axes[0, col]
    ax.plot(times, Bp, '-o', color=BLUE, label='M3 prediction')
    ax.scatter(times, Bo, color='black', zorder=5, label='Observed')
    ax.set_title(dset, fontsize=9)
    ax.set_ylabel('Norm. blood signal')
    ax.text(0.97, 0.97, f'MAE={mae_b:.3f}',
            transform=ax.transAxes, ha='right', va='top', fontsize=8)
    ax.legend(fontsize=8)
    ax = axes[1, col]
    ax.plot(times, Lp, '-s', color=RED, label='M3 prediction')
    ax.scatter(times, Lo, color='darkred', marker='s',
               zorder=5, label='Observed')
    ax.set_xlabel('Time (h)')
    ax.set_ylabel('Norm. liver signal')
    ax.text(0.97, 0.05, f'MAE={mae_l:.3f}',
            transform=ax.transAxes, ha='right', va='bottom', fontsize=8)
    ax.legend(fontsize=8)
plt.tight_layout()
return fig

```

```

def sfig4_krel_reduction(pF, t_long):
    fig, ax = plt.subplots(figsize=(6, 4))
    for frac, col, lbl in [(0.0, BLUE, 'Baseline'),
                           (0.25, GREEN, '-25% k_rel'),
                           (0.50, RED, '-50% k_rel'),
                           (0.75, PURPLE, '-75% k_rel')]:
        p2 = pF.copy(); p2[5] = pF[5] * (1 - frac)
        s = simM3(p2, t_long); L = s.y[2] + s.y[3]
        dur = plateau_duration(L, t_long)
        ax.plot(t_long, L, color=col, lw=2,
                label=f'{lbl} (plateau~{dur:.0f} h)')
    ax.set_xlabel('Time (h)')
    ax.set_ylabel('Hepatic exosome level (a.u.)')
    ax.legend(fontsize=8, title='k_rel modification')
    plt.tight_layout()
    return fig

```

```

# -----
# 14. MAIN
# -----

def main():
    print("=" * 60)
    print("Exosome PK Reservoir Model v3 - Full Analysis")
    print("=" * 60)

    t_obs = choi_time
    bo = choi_blood_mean; lo = choi_liver_mean
    bs = choi_blood_sd; ls = choi_liver_sd
    n_obs = 2 * len(t_obs)

    # --- Fit all three models -----
    print("\n[1/8] Fitting M1, M2, M3 ...")
    rM1 = fit_M1(t_obs, bo, bs)
    rM2 = fit_M2(t_obs, bo, lo, bs, ls)
    rM3 = fit_M3(t_obs, bo, lo, bs, ls)

    pF = P_NOM.copy(); pF[FIT_IDX] = rM3.x
    R_fit = pF[4] / pF[5]
    print(f" M3: k_bind={pF[4]:.4f} k_rel={pF[5]:.4f} R={R_fit:.2f}")

    # --- AICc -----
    print("\n[2/8] AICc model comparison ...")
    ac3 = aicc(resid_M3(rM3.x, P_NOM.copy(), t_obs, bo, lo, bs, ls), 6, n_obs)
    ac2 = aicc(resid_M2(rM2.x, t_obs, bo, lo, bs, ls), 4, n_obs)
    ac1 = aicc(resid_M1(rM1.x, t_obs, bo, bs), 1, len(t_obs))
    print(f" AICc M1={ac1:.1f} M2={ac2:.1f} M3={ac3:.1f}")
    print(f" DAICc M1={ac1-ac3:.1f} M2={ac2-ac3:.1f}")

    # --- Bootstrap -----
    print("\n[3/8] Bootstrap (n=500) ...")
    boots = bootstrap_M3(t_obs, bo, lo, bs, ls, pF, n_boot=500)
    print(f" Successful iterations: {len(boots)}")
    if len(boots) > 10:
        pnames = ['k_clear', 'k_to', 'k_bp', 'k_pb', 'k_bind', 'k_rel']
        for i, nm in enumerate(pnames):
            v = boots[:, i]
            ci = np.percentile(v, [2.5, 97.5])
            cv = np.std(v) / np.mean(v) * 100
            print(f" {nm:8s}: median={np.median(v):.4f} "
                  f"95%CI=[{ci[0]:.4f},{ci[1]:.4f}] CV={cv:.1f}%")

    # --- Sobol -----

```



```

print("\n[4/8] Sobol sensitivity analysis ...")
Si, pnames_sobol = sobol_analysis(pF, n_samples=1024)
for nm, s1, st in zip(pnames_sobol, Si['S1'], Si['ST']):
    print(f"    {nm:8s}: S1={s1:.3f}    ST={st:.3f}")

# — Cross-validation —————
print("\n[5/8] Cross-validation ...")
Bm, Lm = normalised_predict(pF, mirzaaghasi_time)
Bw, Lw = normalised_predict(pF, wiklander_time)
print(f"    Mirzaaghasi: blood MAE={np.mean(np.abs(Bm-mirzaaghasi_blood_norm))}:
        f"liver MAE={np.mean(np.abs(Lm-mirzaaghasi_liver_norm)):.3f}")
print(f"    Wiklander:    blood MAE={np.mean(np.abs(Bw-wiklander_blood_norm)):.3
        f"liver MAE={np.mean(np.abs(Lw-wiklander_liver_norm)):.3f}")

# — Label persistence sensitivity —————
print("\n[6/8] Label persistence sensitivity analysis (Fig 12) ...")
alphas = np.linspace(0.0, 1.0, 9)
R_vals, kb_vals, kr_vals, rmse_vals = label_sensitivity_scan(
    pF, t_obs, bo, lo, bs, ls, alpha_values=alphas, n_restarts=10)
print(f"\n    Summary Table:")
print(f"    {'alpha':>6}    {'R':>6}    {'RMSE':>6}")
for a, Rv, rm in zip(alphas, R_vals, rmse_vals):
    print(f"    {a:>6.2f}    {Rv:>6.2f}    {rm:>6.2f}")
print(f"\n    RMSE minimised at alpha=0: "
        f"{'YES' if np.nanargmin(rmse_vals)==0 else 'NO'}")

# — k_rel reduction —————
print("\n[7/8] k_rel reduction simulations ...")
t_long = np.linspace(0, 120, 400)
for frac in [0.0, 0.25, 0.50, 0.75]:
    p2 = pF.copy(); p2[5] = pF[5] * (1 - frac)
    s = simM3(p2, t_long)
    dur = plateau_duration(s.y[2]+s.y[3], t_long)
    lbl = f"-{int(frac*100)}%" if frac > 0 else "baseline"
    print(f"    k_rel {lbl:>8s}: plateau ~ {dur:.0f} h")

# — Generate figures —————
print("\n[8/8] Generating figures ...")
t_fine = np.linspace(0, 168, 500)
sF = simM3(pF, t_fine)

figs = {
    'Fig01': fig1_blood_fit(t_fine, sF, pF),
    'Fig02': fig2_liver_fit(t_fine, sF),
    'Fig03': fig3_intracellular(t_fine, sF, pF),
    'Fig04': fig4_drug(t_fine, sF),
    'Fig05': fig5_effect(t_fine, sF, pF),

```

```

'Fig06': fig6_model_comparison(t_fine, sF, rM1, rM2),
'Fig07': fig7_aicc(ac1, ac2, ac3),
'Fig08': fig8_bootstrap(boots) if len(boots) > 10 else None,
'Fig09': fig9_sobol(Si, pnames_sobol),
'Fig10': fig10_schematic(),
'Fig11': fig11_R_plateau(pF, t_long),
'Fig12': fig12_label_sensitivity(alphas, R_vals, kb_vals, kr_vals, rmse_v
'SFig01_hepatic_profiles': sfig1_hepatic_profiles(pF, t_long),
'SFig02_2D_heatmap':      sfig2_2d_heatmap(pF, t_long),
'SFig03_crossval':        sfig3_crossval(pF),
'SFig04_krel_reduction':  sfig4_krel_reduction(pF, t_long),
}

for name, fig in figs.items():
    if fig is not None:
        savefig(fig, f'{name}.png')

print("\nDone. All figures saved.")
print("=" * 60)

if __name__ == '__main__':
    main()

```